Skriftlig eksamen i

Algoritmer og datastrukturer

30. maj, 2022

Dette dokument indeholder 5 sider, inklusive forsiden. Der er i alt 6 opgaver.

I programmeringsopgaverne skal du implementere i C++.

Opgave 1 (13 %)

Hvad er Store O (Big-Oh) tidskompleksiteten af `func1`? Begrund dit svar.

```
static int myMethod( int N ){
  int x = 0; int y = 0;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      for (int k = 0; k < N*sqrt(N); k++) {
        x++;
      }
      j *= 2;
    }
    y = i;
  }
  for (int i = 0; i < N*N; i++) {
    y++;
  }
  return x+y;
}
```

Opgave 2 (12 %)

Skriv en **rekursiv** funktion/metode med følgende signatur:

`int oddSquare(int N)`

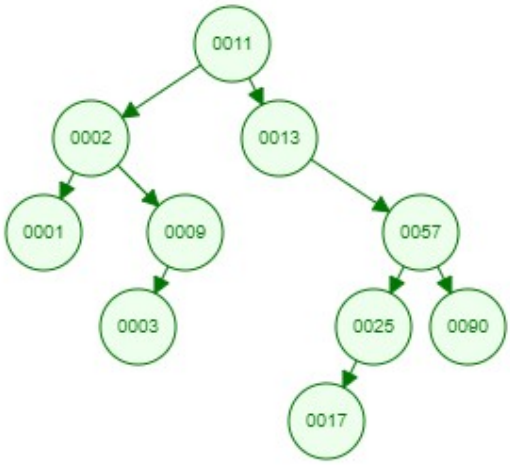
Algoritmen returnerer summen af de ulige tals kvadrattal fra 1 til N. Du kan antage at `N>=0`.

Kaldt med `N = 8` er den korrekte returværdi 84 ($1^2 + 3^2 + 5^2 + 7^2$). Kaldt med `N = 7` er den korrekte returværdi også 84.

Du bedes optimere din metode/algoritme, således at overflødige rekursive kald undgås.

Opgave 3 (20 %)

Nedenstående figur er et binært træ (binary tree)



1. Er det et binært søgetræ – hvorfor/hvorfor ikke?
2. List den rækkefølge, som noderne bliver besøgt i, i en inorder og i en preorder traversering.
3. Det er ikke et AVL-træ – hvorfor ikke? Hvad skal ændres for at det bliver et AVL træ?
4. Kan det have været et AVL træ inden den sidste AVL operation (indsættelse (`insert`) eller sletning (`delete`), men ikke rotation, som er en intern operation)? Eksempler på AVL operationer kunne være `insert(3)` eller `delete(12)`

Opgave 4 (25 %)

1: Skriv en klasse MaxHeap der kan indeholde `int`s ved brug af standardklassen `vector` fra C++ biblioteket `MaxHeap`'en skal have følgende operationer:

```
bool isEmpty() const; //is the heap empty?
int size(); //number of elements in the heap
void insert(const int x); //add an element to the heap
const int findMax() const; // find the maximum element of the heap
int deleteMax(); //delete and return the maximum element of the heap
```

2: Redegør for tidskompleksiteten af de fem operationer

`vector` har følgende operationer som du kan bruge (se næste side). Hvis du skal gennemløbe `vector`'en, skal du bruge en iterator. Iterator typen erklæres på følgende måde: `std::vector<int>::iterator` og har metoderne `begin()`, `end()`, `++` samt `*` (som virker som beskrevet i undervisningen)

Iterators

`begin` `cbegin` (C++11)	returns an iterator to the beginning (public member function)
`end` `cend` (C++11)	returns an iterator to the end (public member function)
`rbegin` `crbegin` (C++11)	returns a reverse iterator to the beginning (public member function)
`rend` `crend` (C++11)	returns a reverse iterator to the end (public member function)

Capacity

`empty`	checks whether the container is empty (public member function)
`size`	returns the number of elements (public member function)
`max_size`	returns the maximum possible number of elements (public member function)
`reserve`	reserves storage (public member function)
`capacity`	returns the number of elements that can be held in currently allocated storage (public member function)
`shrink_to_fit` (C++11)	reduces memory usage by freeing unused memory (public member function)

Modifiers

`clear`	clears the contents (public member function)
`insert`	inserts elements (public member function)
`emplace` (C++11)	constructs element in-place (public member function)
`erase`	erases elements (public member function)
`push_back`	adds an element to the end (public member function)
`emplace_back` (C++11)	constructs an element in-place at the end (public member function)
`pop_back`	removes the last element (public member function)
`resize`	changes the number of elements stored (public member function)
`swap`	swaps the contents (public member function)

The modifiers that require a parameter, takes an `Iterator` as argument

Opgave 5 (20 %)

Et (balanceret) binært søgetræ (binary search tree) kan implementeres som et simpelt array af heltal, hvor det første element (index 0) er ubrugt af hensyn til adressering. -1 angiver at elementet ikke findes/er tomt.

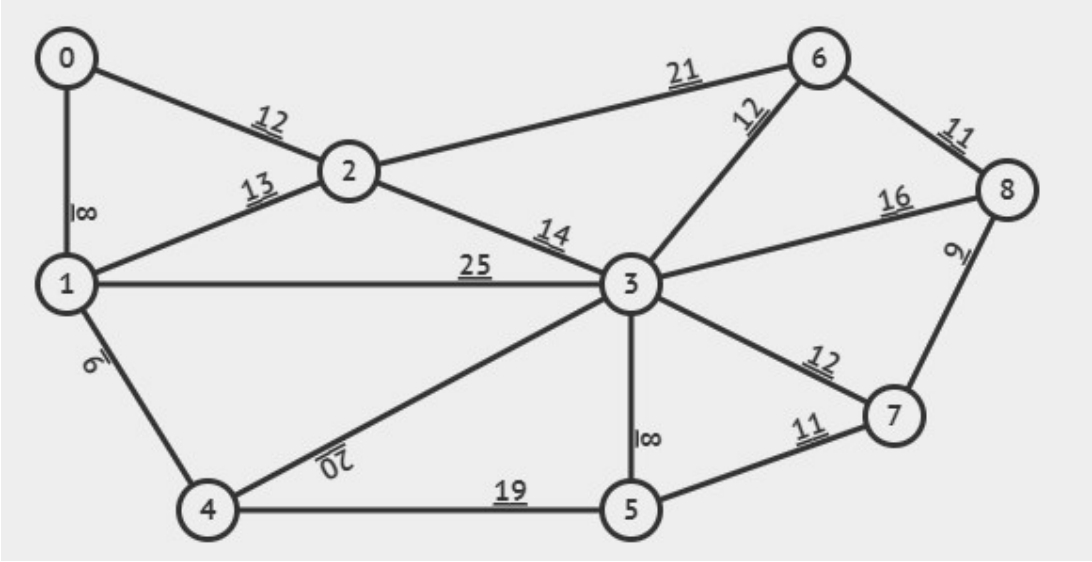
1. Beskriv hvorledes dette kan gøres effektivt (således at et givet element kan findes i O(log(N) forudsat det binære søgetræ er balanceret), hvor N er antallet af elementer i træet.
2. Kan det nedenstående array repræsentere et binært søgetræ? Begrund dit svar:

	27	21	50	17	23	48	-1	80	19	-1	-1	49	46	75	90
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

3. Hvis arrayet ikke kan repræsentere et binært søgetræ, så modificer det så det kan og argumenter for hvorfor det er tilfældet, samt tegn det binære søgetræ det repræsenterer
4. Hvilken træ gennemløbs algoritme kan bruges for at komme fra det binære træ til array implementeringen (dvs besøger knuderne i samme rækkefølge som de findes i arrayet)? Hvad er tidskompleksiteten af dette gennemløb?

Opgave 6 (10 %)

Givet følgende graf:



Hvorledes vil det mindste udspændende træ (minimum spanning tree) komme til at se ud hvis vi anvender Kruskal's algoritme. Beskriv den rækkefølge hvorved kanterne tilføjes.